

ESPECIFICAÇÃO DE REQUISITOS E PROJETO DE SOFTWARE

Plataforma Escambo - MVP Web Mobile-First

Status do Projeto: Em Planejamento e Design

Versão da Documentação: 1.0 (Consolidada para MVP)

Público-Alvo Tecnológico: Desenvolvedores e Idealizadores do Projeto

Stack Principal: React, Next.js, Tailwind, FastAPI, SQLAlchemy, PostgreSQL

Setembro, 2026
Nordeste, Brasil

1. INTRODUÇÃO E VISÃO GERAL DO PROJETO

O projeto Escambo é uma plataforma digital dedicada exclusivamente à facilitação de permutas e trocas diretas de bens e mercadorias, operando sem intermediação financeira obrigatória. Diferenciando-se das ferramentas de classificados genéricos como a OLX, o foco do Escambo reside na construção de uma comunidade onde a moeda de troca é o próprio produto do outro usuário. O projeto nasce com um apelo regionalizado e focado, tendo como marco inicial de validação o mercado do Nordeste brasileiro, com destaque para a inclusão de categorias de comércio rústico e agrícola (animais rurais como burros e cavalos, carroças, selas, arreios, sementes e implementos rurais), além de produtos convencionais de tecnologia, eletrodomésticos e utilitários.

Sob a ótica da Engenharia de Software Ágil, a estratégia do projeto de vida do Escambo foi delimitada para priorizar uma aplicação Web Mobile-First na versão v1 (MVP), adiando a construção de aplicativos móveis nativos (em Kotlin/Swift) para etapas posteriores de escala. Essa decisão reduz drasticamente o tempo de entrega (Time-to-Market), elimina as restrições burocráticas e custos de comissão das lojas de aplicativos (Google Play Store e Apple App Store) e foca na validação da proposta de valor diretamente no navegador de qualquer dispositivo celular moderno por meio de links de carregamento instantâneo.

2. ESPECIFICAÇÃO DE REQUISITOS DO PROJETO

A seguir, são detalhados os Requisitos Funcionais (RF), Requisitos Não Funcionais (RNF) e Requisitos de Sistema (RS) que governam o desenvolvimento da plataforma.

2.1 Atores do Sistema

Ator	Descrição e Responsabilidade no Sistema
Usuário Comum / Visitante	Acessa a plataforma de forma pública, visualiza anúncios, pesquisa por categorias e lê artigos de segurança, sem necessidade de autenticação.
Usuário Autenticado	Cadastra dados obrigatórios (Nome, CPF, E-mail, Telefone), aceita os termos da LGPD, publica ofertas de troca, envia propostas em anúncios de terceiros e interage via chat interno seguro.
Administrador do Sistema	Responsável pela moderação de anúncios impróprios, análise de denúncias de fraudes, suporte a usuários bloqueados e gerenciamento de categorias agrícolas e urbanas.

2.2 Requisitos Funcionais (RF)

ID	Requisito Funcional	Detalhamento e Regras de Negócio
RF01	Cadastro com Consentimento LGPD	O sistema deve coletar Nome Completo, CPF, E-mail e Telefone. É mandatório o aceite inequívoco de um termo de consentimento descrevendo a finalidade de segurança antes de permitir o acesso.
RF02	Publicação de Anúncio de Escambo	O usuário cadastrado poderá publicar um anúncio informando título, descrição técnica rústica (ex: idade do burro, estado da carroça), fotos, categoria e, obrigatoriamente, descrever em texto livre 'O que aceita em troca'.
RF03	Busca e Filtragem Geográfica	Exibição de um Hub principal Mobile-First onde o usuário filtra itens por categorias agrícolas ou urbanas, raio de distância ou cidade/estado específica do Nordeste.
RF04	Proposta de Troca Direta	O usuário interessado envia uma proposta contendo a descrição de sua oferta (podendo vincular um anúncio próprio ou propor uma permuta direta). O dono do anúncio original recebe uma notificação.
RF05	Chat Interno Bidirecional	Uma vez iniciada uma proposta

	Seguro	de escambo, um canal de chat privado é aberto. Informações de contato direto (telefone/e-mail) permanecem ocultas até que ambos deem aceite formal na troca.
RF06	Validação de Identidade e CPF	O sistema deve verificar o formato do CPF no cadastro. Em segundo plano, armazena o hash criptográfico do CPF para conformidade com o Privacy by Design.

2.3 Requisitos Não Funcionais (RNF)

ID	Requisito Não Funcional	Métrica de Aceitação e Qualidade
RNF01	Desempenho de Carregamento (Performance)	O sistema frontend construído em Next.js (com Server-Side Rendering) deve carregar a página de detalhe do produto em menos de 1.8 segundos sob conexão de rede móvel 3G/4G no interior do Nordeste.
RNF02	Usabilidade Mobile-First (ISO 9241-11)	A interface web deve ser inteiramente adaptável a telas móveis (responsiva) de baixa resolução, mantendo os botões de ação ('Propor Troca', 'Ler Alerta') acessíveis com uma única mão.
RNF03	Segurança e Proteção de Dados (LGPD)	Todos os dados pessoais devem ser trafegados em conexões seguras HTTPS. O CPF deve ser guardado no

PostgreSQL com criptografia simétrica AES-256 e os e-mails e telefones devem ser ofuscados no banco secundário.

RNF04

Auditoria e Logs de Consentimento

Cada registro de consentimento dos termos de uso da LGPD deve gerar um log criptográfico não editável contendo: ID do Usuário, Versão do Termo, Timestamp e IP de cadastro.

3. SEGURANÇA, TRANSPARÊNCIA (LGPD) E PREVENÇÃO DE FRAUDES

Como a plataforma lida diretamente com dados pessoais sensíveis (Nome, CPF, E-mail e Telefone) para validar a segurança das transações, a arquitetura deve aplicar rígidos mecanismos de segurança cibernética e de governança organizados de acordo com a legislação brasileira (LGPD).

3.1 Fluxo de Tratamento de Dados (Privacy by Design)

Os seguintes controles técnicos serão implementados no código e banco de dados:

- **Segregação de Dados Pessoais:** Informações pessoais de cadastro (CPF e Telefone) serão mantidas em tabelas isoladas do PostgreSQL e nunca trafegarão em payloads públicos da API.
- **Criptografia Assimétrica e Hashing:** CPFs serão criptografados antes de serem inseridos no banco usando o algoritmo Fernet (biblioteca cryptography do Python). Senhas serão criptografadas via bcrypt com salt único.
- **Log de Consentimento LGPD (Auditorias):** Toda vez que o usuário aceita uma nova versão do termo legal, uma linha é criada em um ledger imutável de logs de consentimento no PostgreSQL para provar a conformidade jurídica em auditorias.

3.2 Termos Legais Legíveis e Prevenção de Fraudes Agrícolas

A plataforma Escambo atua de forma diferenciada devido às trocas de animais (burros, cavalos, bovinos) e implementos agrícolas no interior. Como o trâmite desses animais é regulado por órgãos estaduais de defesa agropecuária, a plataforma conterà alertas visuais instrutivos e regras claras integradas às interfaces de IHC:



ALERTA DE SEGURANÇA E PREVENÇÃO A GOLPES (INTEGRADO NO CHAT):

1. Trocas Presenciais: Marque as trocas de mercadorias ou animais em locais públicos e de grande movimento, como feiras livres de gado da sua cidade ou praças centrais.
2. Jamais Pague Sinais ou Fretes Adiantados: Desconfie se a outra parte exigir depósitos de frete em dinheiro ou PIX antes de você validar pessoalmente a mercadoria ou o estado de saúde do animal.
3. Guia de Trânsito Animal (GTA): Lembre-se que o trânsito legal de animais exige a emissão da GTA expedida pelo órgão de defesa agropecuária local. Garanta que o animal possui carteira de vacinação atualizada.

4. PROJETO ARQUITETURAL E MODELO DE BANCO DE DADOS

Para acelerar o desenvolvimento de forma organizada e garantir a rastreabilidade e integridade das regras do software, é apresentado a seguir o design lógico do banco de dados relacional (PostgreSQL) mapeado diretamente para o framework SQLAlchemy do Flask/FastAPI (ORM). Este mapeamento servirá como a base estrutural para a programação imediata do back-end do Escambo.

Script SQLAlchemy de Produção (Modelos de Dados do Escambo):

```
import datetime
from enum import Enum
from sqlalchemy import Column, Integer, String, Text, DateTime, ForeignKey, Enum as
SqlEnum, Boolean
from sqlalchemy.orm import declarative_base, relationship

Base = declarative_base()

class CategoriaEnum(Enum):
    ANIMAIS = "Animais"
    CARROCAS_SELAS = "Carroças e Selas"
    TECNOLOGIA = "Tecnologia"
    VARIADOS = "Variados"

class StatusAnuncioEnum(Enum):
    ATIVO = "Ativo"
    TROCADO = "Trocado"
    INATIVO = "Inativo"

class StatusPropostaEnum(Enum):
    PENDENTE = "Pendente"
    ACEITA = "Aceita"
    RECUSADA = "Recusada"

class Usuario(Base):
    __tablename__ = 'usuarios'

    id = Column(Integer, primary_key=True, autoincrement=True)
    nome_completo = Column(String(100), nullable=False)
    cpf_criptografado = Column(String(255), nullable=False, unique=True) # Criptografado
    com Fernet
    email = Column(String(100), nullable=False, unique=True)
    telefone = Column(String(20), nullable=False) # Para envio do SMS de validação
    senha_hash = Column(String(128), nullable=False)
    created_at = Column(DateTime, default=datetime.datetime.utcnow)

    anuncios = relationship("Anuncio", back_populates="usuario")
    consentimentos = relationship("LogConsentimento", back_populates="usuario")

class LogConsentimento(Base):
    __tablename__ = 'logs_consentimento_lgpd'

    id = Column(Integer, primary_key=True, autoincrement=True)
    usuario_id = Column(Integer, ForeignKey('usuarios.id'), nullable=False)
    versao_termos = Column(String(10), nullable=False, default="v1.0")
    data_aceite = Column(DateTime, default=datetime.datetime.utcnow)
    ip_origem = Column(String(45), nullable=True) # Suporta IPv4 e IPv6

    usuario = relationship("Usuario", back_populates="consentimentos")

class Anuncio(Base):
    __tablename__ = 'anuncios'

    id = Column(Integer, primary_key=True, autoincrement=True)
```



```

    usuario_id = Column(Integer, ForeignKey('usuarios.id'), nullable=False)
    titulo = Column(String(100), nullable=False)
    descricao = Column(Text, nullable=False) # Ex: idade do cavalo, estado do arreio
    categoria = Column(SqlEnum(CategoriaEnum), nullable=False)
    o_que_aceita_em_troca = Column(Text, nullable=False) # Requisito obrigatório de
escopo
    cidade = Column(String(100), nullable=False) # Ex: Caruaru, Feira de Santana
    estado = Column(String(2), nullable=False) # Ex: PE, BA, CE, PB
    status = Column(SqlEnum(StatusAnuncioEnum), default=StatusAnuncioEnum.ATIVO)
    created_at = Column(DateTime, default=datetime.datetime.utcnow)

    usuario = relationship("Usuario", back_populates="anuncios")
    propostas_recebidas = relationship("PropostaTroca",
foreign_keys="[PropostaTroca.anuncio_origem_id]")

class PropostaTroca(Base):
    __tablename__ = 'propostas_troca'

    id = Column(Integer, primary_key=True, autoincrement=True)
    anuncio_origem_id = Column(Integer, ForeignKey('anuncios.id'), nullable=False)
    anuncio_proposto_id = Column(Integer, ForeignKey('anuncios.id'), nullable=True) # Se
propôs anúncio dele
    usuario_proponente_id = Column(Integer, ForeignKey('usuarios.id'), nullable=False)
    proposta_descricao = Column(Text, nullable=False) # Mensagem ou termos da permuta
    status = Column(SqlEnum(StatusPropostaEnum), default=StatusPropostaEnum.PENDENTE)
    created_at = Column(DateTime, default=datetime.datetime.utcnow)

    conversas = relationship("MensagemChat", back_populates="proposta")

class MensagemChat(Base):
    __tablename__ = 'mensagens_chat'

    id = Column(Integer, primary_key=True, autoincrement=True)
    proposta_id = Column(Integer, ForeignKey('propostas_troca.id'), nullable=False)
    remetente_id = Column(Integer, ForeignKey('usuarios.id'), nullable=False)
    texto_mensagem = Column(Text, nullable=False)
    enviado_em = Column(DateTime, default=datetime.datetime.utcnow)

    proposta = relationship("PropostaTroca", back_populates="conversas")

```

5. PLANO DE SPRINT E IMPLEMENTAÇÃO ÁGIL

Para manter o ritmo focado em entregas de incrementos úteis e utilizáveis conforme a filosofia do Manifesto Ágil, o desenvolvimento do MVP do Escambo será gerenciado por meio de um quadro Kanban de ritmo dinâmico. O quadro abaixo organiza os pacotes de trabalho priorizados para o primeiro ciclo quinzenal de desenvolvimento.

5.1 Quadro Kanban - Sprint 1 (MVP)

Coluna	Atividade Mapeada	Critério de Aceitação / Status
A Fazer	Setup do Banco PostgreSQL e ORM SQLAlchemy	Conexão de migrations Flask-Migrate ou FastAPI alembic funcionando 100% com tabelas criptografadas de CPF.
A Fazer	Fluxo de Consentimento e Registro de Login	Exibição obrigatória do modal da LGPD e inserção correta de log_consentimento associado ao IP do usuário.
Em Progresso	API de Cadastro e Moderação de Anúncios	Mapeamento das rotas FastAPI com listagem inteligente por geolocalização e distinção clara de categorias agrícolas (animais, selas).
Em Teste	Componentização Responsiva (Next.js/Tailwind)	Páginas públicas dos anúncios carregando com layout limpo e responsivo adaptado para conexões instáveis.
Concluído	Especificação de Requisitos e Termos de Uso v1.0	Documentação de engenharia refinada e validada para ser anexada ao repositório central do projeto.